

AI Analytics - SQL Evaluation Report

Generated: 2026-06-02 14:23 | Total prompts: 31

Total Prompts	Passed	Failed	Errors
31	25	5	1

Q1: For each product category, find the single product that generated the highest total revenue. Show the category name, product ID, and its total revenue.

PASS

Proposed SQL (Expected):

```
WITH product_revenue AS (SELECT p.category_id, p.product_id, SUM(oi.price * oi.qty) AS total_revenue FROM products p JOIN
order_items oi ON p.product_id = oi.product_id GROUP BY p.category_id, p.product_id), ranked AS (SELECT pr.category_id,
pr.product_id, pr.total_revenue, ROW_NUMBER() OVER (PARTITION BY pr.category_id ORDER BY pr.total_revenue DESC) AS rn FROM
product_revenue pr) SELECT c.category_name, r.product_id, r.total_revenue FROM ranked r JOIN categories c ON r.category_id =
c.category_id WHERE r.rn = 1
```

Proposed SQL Output (sorted):

category_name	product_id	total_revenue

Cat_1	1868	566880.0
Cat_10	6302	600997.0
Cat_2	6932	558276.0
Cat_3	9133	620725.0
Cat_4	5201	610779.0
Cat_5	9318	561237.0
Cat_6	4015	556797.0
Cat_7	7784	551247.0
Cat_8	9952	566549.0
Cat_9	1907	587438.0
... and 20 more rows		
(Total rows: 30)		

Generated SQL (AI Output):

```
WITH ProductRevenue AS (
SELECT
p.category_id,
p.product_id,
SUM(oi.qty * oi.price) AS total_revenue
FROM order_items AS oi
JOIN products AS p
ON oi.product_id = p.product_id
GROUP BY
p.category_id,
p.product_id
), RankedProducts AS (
SELECT
category_id,
product_id,
total_revenue,
ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY total_revenue DESC) AS rn
FROM ProductRevenue
)
SELECT
c.category_name,
rp.product_id,
rp.total_revenue
FROM RankedProducts AS rp
JOIN categories AS c
ON rp.category_id = c.category_id
WHERE
rp.rn = 1
ORDER BY
c.category_name;
```

Generated SQL Output (sorted):

category_name	product_id	total_revenue

Cat_1	1868	566880.0
Cat_10	6302	600997.0
Cat_11	6129	538196.0
Cat_12	1584	598328.0
Cat_13	5485	619073.0
Cat_14	2089	562508.0
Cat_15	8933	558342.0
Cat_16	6388	581943.0
Cat_17	2123	562821.0
Cat_18	745	674395.0
... and 20 more rows		
(Total rows: 30)		

Q2: Calculate the month-over-month percentage change in total number of orders for the year 2023. Show the month, order count, and growth percentage compared to the previous month.

ERROR

Proposed SQL (Expected):

```
WITH monthly_orders AS (SELECT EXTRACT(MONTH FROM order_date) AS month, COUNT(order_id) AS order_count FROM orders WHERE
EXTRACT(YEAR FROM order_date) = 2023 GROUP BY month) SELECT month, order_count, ROUND(100.0 * (order_count - LAG(order_count)
OVER (ORDER BY month)) / NULLIF(LAG(order_count) OVER (ORDER BY month), 0), 2) AS pct_change FROM monthly_orders ORDER BY
month
```

Proposed SQL Output (sorted):

```
month | order_count | pct_change
-----
1.0 | 6369.0 | nan
10.0 | 6350.0 | 2.98
2.0 | 5769.0 | -9.42
3.0 | 6402.0 | 10.97
4.0 | 6142.0 | -4.06
5.0 | 6450.0 | 5.01
6.0 | 6136.0 | -4.87
7.0 | 6369.0 | 3.8
8.0 | 6344.0 | -0.39
9.0 | 6166.0 | -2.81
... and 2 more rows
(Total rows: 12)
```

Generated SQL (AI Output):

```
WITH MonthlyOrders AS (
SELECT
EXTRACT(MONTH FROM order_date) AS month,
COUNT(order_id) AS order_count
FROM
orders
WHERE
EXTRACT(YEAR FROM order_date) = 2023
GROUP BY
EXTRACT(MONTH FROM order_date)
ORDER BY
month
)
SELECT
month,
order_count,
LAG(order_count, 1, 0) OVER (ORDER BY month) AS previous_month_orders,
ROUND(
(order_count - LAG(order_count, 1, 0) OVER (ORDER BY month)) * 100.0 / LAG(order_count, 1, 0) OVER (ORDER BY month),
2
) AS mom_growth_percentage
FROM
MonthlyOrders;
```

Generated SQL Output (sorted):

```
ERROR: (psycpg2.errors.DivisionByZero) division by zero

(order_count - LAG(order_count, 1, 0) OVER (ORDER BY month)) * 100.0 / LAG(order_count, 1, 0) OVER (ORDER BY month),
2
COUNT(order_id) AS order_count
EXTRACT(MONTH FROM order_date)
EXTRACT(MONTH FROM order_date) AS month,
EXTRACT(YEAR FROM order_date) = 2023
month
orders
) AS mom_growth_percentage
FROM
GROUP BY
LAG(order_count, 1, 0) OVER (ORDER BY month) AS previous_month_orders,
MonthlyOrders;]
ORDER BY
ROUND(
SELECT
WHERE
month,
order_count,
(Background on this error at: https://sqlalche.me/e/20/9h9h)
)
FROM
SELECT
[SQL: WITH MonthlyOrders AS (
```

Q3: Find the top 5 customers by net revenue, where net revenue is total payments minus total refunds.
Show customer ID, city, total payments, total refunds, and net revenue.

PASS

Proposed SQL (Expected):

```
WITH customer_payments AS (SELECT o.customer_id, SUM(pay.amount) AS total_paid FROM orders o JOIN payments pay ON o.order_id = pay.order_id GROUP BY o.customer_id), customer_refunds AS (SELECT o.customer_id, SUM(r.refund) AS total_refunded FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN returns r ON oi.order_item_id = r.order_item_id GROUP BY o.customer_id) SELECT c.customer_id, c.city, COALESCE(cp.total_paid, 0) AS total_payments, COALESCE(cr.total_refunded, 0) AS total_refunds, COALESCE(cp.total_paid, 0) - COALESCE(cr.total_refunded, 0) AS net_revenue FROM customers c LEFT JOIN customer_payments cp ON c.customer_id = cp.customer_id LEFT JOIN customer_refunds cr ON c.customer_id = cr.customer_id ORDER BY net_revenue DESC LIMIT 5
```

Proposed SQL Output (sorted):

customer_id	city	total_payments	total_refunds	net_revenue
25392	Bangalore	188147.0	0.0	188147.0
33455	Delhi	208559.0	10515.0	198044.0
38809	Bangalore	188362.0	0.0	188362.0
42251	Delhi	196566.0	4003.0	192563.0
45285	Pune	184354.0	0.0	184354.0

(Total rows: 5)

Generated SQL (AI Output):

```
WITH customer_payments AS (
SELECT
o.customer_id,
SUM(p.amount) AS total_payments
FROM payments AS p
JOIN orders AS o
ON p.order_id = o.order_id
GROUP BY
o.customer_id
), customer_refunds AS (
SELECT
o.customer_id,
SUM(r.refund) AS total_refunds
FROM returns AS r
JOIN order_items AS oi
ON r.order_item_id = oi.order_item_id
JOIN orders AS o
ON oi.order_id = o.order_id
GROUP BY
o.customer_id
)
SELECT
c.customer_id,
c.city,
COALESCE(cp.total_payments, 0) AS total_payments,
COALESCE(cr.total_refunds, 0) AS total_refunds,
(
COALESCE(cp.total_payments, 0) - COALESCE(cr.total_refunds, 0)
) AS net_revenue
FROM customers AS c
LEFT JOIN customer_payments AS cp
ON c.customer_id = cp.customer_id
LEFT JOIN customer_refunds AS cr
ON c.customer_id = cr.customer_id
ORDER BY
net_revenue DESC
LIMIT 5;
```

Generated SQL Output (sorted):

customer_id	city	total_payments	total_refunds	net_revenue
25392	Bangalore	188147.0	0.0	188147.0
33455	Delhi	208559.0	10515.0	198044.0
38809	Bangalore	188362.0	0.0	188362.0
42251	Delhi	196566.0	4003.0	192563.0
45285	Pune	184354.0	0.0	184354.0

(Total rows: 5)

Q4: Find all product categories where the return rate (number of returned order items divided by total items sold) exceeds 10%. Show category name, total items sold, total returns, and return rate percentage.

PASS

Proposed SQL (Expected):

```
SELECT c.category_name, SUM(oi.qty) AS total_items_sold, COUNT(r.return_id) AS total_returns, ROUND(100.0 * COUNT(r.return_id) / NULLIF(SUM(oi.qty), 0), 2) AS return_rate_pct FROM categories c JOIN products p ON c.category_id = p.category_id JOIN order_items oi ON p.product_id = oi.product_id LEFT JOIN returns r ON oi.order_item_id = r.order_item_id GROUP BY c.category_name HAVING ROUND(100.0 * COUNT(r.return_id) / NULLIF(SUM(oi.qty), 0), 2) > 10 ORDER BY return_rate_pct DESC
```

Proposed SQL Output (sorted):

(0 rows returned)

Generated SQL (AI Output):

```
WITH CategorySales AS (
SELECT
c.category_name,
SUM(oi.qty) AS total_items_sold
FROM order_items AS oi
JOIN products AS p ON oi.product_id = p.product_id
JOIN categories AS c ON p.category_id = c.category_id
GROUP BY c.category_name
),
CategoryReturns AS (
SELECT
c.category_name,
SUM(oi.qty) AS total_items_returned
FROM returns AS r
JOIN order_items AS oi ON r.order_item_id = oi.order_item_id
JOIN products AS p ON oi.product_id = p.product_id
JOIN categories AS c ON p.category_id = c.category_id
GROUP BY c.category_name
)
SELECT
cs.category_name,
cs.total_items_sold,
COALESCE(cr.total_items_returned, 0) AS total_returns,
(COALESCE(cr.total_items_returned, 0.0) / cs.total_items_sold) * 100 AS return_rate_percentage
FROM CategorySales AS cs
LEFT JOIN CategoryReturns AS cr ON cs.category_name = cr.category_name
WHERE (COALESCE(cr.total_items_returned, 0.0) / cs.total_items_sold) > 0.10
ORDER BY return_rate_percentage DESC;
```

Generated SQL Output (sorted):

(0 rows returned)

Q5: Rank all stores by their total revenue. For each store show the city, total revenue, rank, and what percentage of the overall total revenue that store contributes.

PASS

Proposed SQL (Expected):

```
WITH store_revenue AS (SELECT s.store_id, s.city, SUM(oi.price * oi.qty) AS total_revenue FROM stores s JOIN orders o ON
s.store_id = o.store_id JOIN order_items oi ON o.order_id = oi.order_id GROUP BY s.store_id, s.city), total AS (SELECT
SUM(total_revenue) AS grand_total FROM store_revenue) SELECT sr.city, sr.total_revenue, RANK() OVER (ORDER BY sr.total_revenue
DESC) AS revenue_rank, ROUND(100.0 * sr.total_revenue / t.grand_total, 2) AS revenue_pct FROM store_revenue sr, total t ORDER
BY revenue_rank
```

Proposed SQL Output (sorted):

```
city | total_revenue | revenue_rank | revenue_pct
-----
Bangalore | 39794678.0 | 9 | 1.04
Delhi | 39833700.0 | 7 | 1.04
Delhi | 40093705.0 | 1 | 1.05
Mumbai | 39635295.0 | 10 | 1.04
Mumbai | 39830349.0 | 8 | 1.04
Mumbai | 39849329.0 | 6 | 1.04
Mumbai | 39929457.0 | 4 | 1.04
Mumbai | 40030373.0 | 3 | 1.05
Pune | 39851073.0 | 5 | 1.04
Pune | 40082772.0 | 2 | 1.05
... and 90 more rows
(Total rows: 100)
```

Generated SQL (AI Output):

```
WITH StoreRevenue AS (
SELECT
s.store_id,
s.city,
SUM(oi.qty * oi.price) AS total_revenue
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN stores s ON o.store_id = s.store_id
GROUP BY s.store_id, s.city
)
SELECT
s.city,
CAST(s.total_revenue AS NUMERIC(10, 2)) as total_revenue,
RANK() OVER (ORDER BY s.total_revenue DESC) AS store_rank,
CAST((s.total_revenue / SUM(s.total_revenue) OVER ()) * 100 AS NUMERIC(5, 2)) AS percentage_of_total
FROM StoreRevenue s
ORDER BY store_rank;
```

Generated SQL Output (sorted):

```
city | total_revenue | store_rank | percentage_of_total
-----
Bangalore | 39794678.0 | 9 | 1.04
Delhi | 39833700.0 | 7 | 1.04
Delhi | 40093705.0 | 1 | 1.05
Mumbai | 39635295.0 | 10 | 1.04
Mumbai | 39830349.0 | 8 | 1.04
Mumbai | 39849329.0 | 6 | 1.04
Mumbai | 39929457.0 | 4 | 1.04
Mumbai | 40030373.0 | 3 | 1.05
Pune | 39851073.0 | 5 | 1.04
Pune | 40082772.0 | 2 | 1.05
... and 90 more rows
(Total rows: 100)
```

Q6: Find all customers who have exclusively placed orders that had a promotion applied — meaning they have never placed an order without a promotion. Show customer ID and city.

PASS

Proposed SQL (Expected):

```
SELECT c.customer_id, c.city FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id AND o.promotion_id IS NULL) AND EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
```

Proposed SQL Output (sorted):

```
customer_id | city
-----
10295 | Delhi
11890 | Delhi
2026 | Mumbai
22262 | Mumbai
2848 | Mumbai
36761 | Mumbai
39118 | Pune
39665 | Pune
43007 | Mumbai
46770 | Delhi
... and 49876 more rows
(Total rows: 49886)
```

Generated SQL (AI Output):

```
SELECT
c.customer_id,
c.city
FROM
customers AS c
JOIN
orders AS o ON c.customer_id = o.customer_id
GROUP BY
c.customer_id, c.city
HAVING
COUNT(o.order_id) = COUNT(o.promotion_id);
```

Generated SQL Output (sorted):

```
customer_id | city
-----
11233 | Bangalore
26264 | Bangalore
35634 | Pune
37876 | Mumbai
40694 | Bangalore
42756 | Bangalore
44127 | Bangalore
47051 | Bangalore
47114 | Bangalore
5468 | Pune
... and 49876 more rows
(Total rows: 49886)
```

Q7: For each supplier country, calculate total revenue and total refunds. Show only countries where total refunds exceed 20% of total revenue. Display country, total revenue, total refunds, and refund ratio as a percentage.

PASS

Proposed SQL (Expected):

```
WITH supplier_stats AS (SELECT s.country, SUM(oi.price * oi.qty) AS total_revenue, COALESCE(SUM(r.refund), 0) AS total_refunds
FROM suppliers s JOIN products p ON s.supplier_id = p.supplier_id JOIN order_items oi ON p.product_id = oi.product_id LEFT
JOIN returns r ON oi.order_item_id = r.order_item_id GROUP BY s.country) SELECT country, total_revenue, total_refunds,
ROUND(100.0 * total_refunds / NULLIF(total_revenue, 0), 2) AS refund_ratio_pct FROM supplier_stats WHERE total_refunds > 0.20
* total_revenue ORDER BY refund_ratio_pct DESC
```

Proposed SQL Output (sorted):

(0 rows returned)

Generated SQL (AI Output):

```
WITH CountrySales AS (
SELECT
s.country,
SUM(oi.price * oi.qty) AS total_revenue,
SUM(r.refund) AS total_refunds
FROM suppliers AS s
JOIN products AS p
ON s.supplier_id = p.supplier_id
JOIN order_items AS oi
ON p.product_id = oi.product_id
LEFT JOIN returns AS r
ON oi.order_item_id = r.order_item_id
GROUP BY
s.country
)
SELECT
country,
total_revenue,
total_refunds,
(total_refunds / total_revenue) * 100 AS refund_ratio_percentage
FROM CountrySales
WHERE
total_refunds > 0.20 * total_revenue
ORDER BY
refund_ratio_percentage DESC;
```

Generated SQL Output (sorted):

(0 rows returned)

Q8: Divide all employees in each store city into 4 salary quartiles (1 = lowest, 4 = highest) within their city. Show each employee's ID, store city, salary, and their salary quartile.

PASS

Proposed SQL (Expected):

```
SELECT e.employee_id, s.city, e.salary, NTILE(4) OVER (PARTITION BY s.city ORDER BY e.salary) AS salary_quartile FROM
employees e JOIN stores s ON e.store_id = s.store_id ORDER BY s.city, salary_quartile, e.salary
```

Proposed SQL Output (sorted):

employee_id	city	salary	salary_quartile
151	Bangalore	20741.0	1
235	Bangalore	22556.0	1
342	Bangalore	21958.0	1
357	Bangalore	20695.0	1
570	Bangalore	21829.0	1
662	Bangalore	22468.0	1
80	Bangalore	20078.0	1
871	Bangalore	20663.0	1
886	Bangalore	21839.0	1
921	Bangalore	22748.0	1
... and 990 more rows			
(Total rows: 1000)			

Generated SQL (AI Output):

```
SELECT
e.employee_id,
s.city,
e.salary,
NTILE(4) OVER (PARTITION BY s.city ORDER BY e.salary ASC) AS salary_quartile
FROM
employees AS e
JOIN
stores AS s ON e.store_id = s.store_id
ORDER BY
s.city,
salary_quartile,
e.salary;
```

Generated SQL Output (sorted):

employee_id	city	salary	salary_quartile
151	Bangalore	20741.0	1
235	Bangalore	22556.0	1
342	Bangalore	21958.0	1
357	Bangalore	20695.0	1
570	Bangalore	21829.0	1
662	Bangalore	22468.0	1
80	Bangalore	20078.0	1
871	Bangalore	20663.0	1
886	Bangalore	21839.0	1
921	Bangalore	22748.0	1
... and 990 more rows			
(Total rows: 1000)			

Q9: Calculate the cumulative running total of orders placed each month in 2023. Show the month number, the count of orders that month, and the running cumulative total up to that month.

PASS

Proposed SQL (Expected):

```
WITH monthly AS (SELECT EXTRACT(MONTH FROM order_date) AS month, COUNT(order_id) AS monthly_count FROM orders WHERE EXTRACT(YEAR FROM order_date) = 2023 GROUP BY month) SELECT month, monthly_count, SUM(monthly_count) OVER (ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM monthly ORDER BY month
```

Proposed SQL Output (sorted):

month	monthly_count	running_total
1.0	6369.0	6369.0
10.0	6350.0	62497.0
2.0	5769.0	12138.0
3.0	6402.0	18540.0
4.0	6142.0	24682.0
5.0	6450.0	31132.0
6.0	6136.0	37268.0
7.0	6369.0	43637.0
8.0	6344.0	49981.0
9.0	6166.0	56147.0
... and 2 more rows		
(Total rows: 12)		

Generated SQL (AI Output):

```
WITH monthly_orders AS (  
  SELECT  
    EXTRACT(MONTH FROM order_date) AS month_number,  
    COUNT(order_id) AS monthly_order_count  
  FROM  
    orders  
  WHERE  
    EXTRACT(YEAR FROM order_date) = 2023  
  GROUP BY  
    EXTRACT(MONTH FROM order_date)  
)  
SELECT  
  month_number,  
  monthly_order_count,  
  SUM(monthly_order_count) OVER (ORDER BY month_number) AS running_total  
FROM  
  monthly_orders  
ORDER BY  
  month_number;
```

Generated SQL Output (sorted):

month_number	monthly_order_count	running_total
1.0	6369.0	6369.0
10.0	6350.0	62497.0
2.0	5769.0	12138.0
3.0	6402.0	18540.0
4.0	6142.0	24682.0
5.0	6450.0	31132.0
6.0	6136.0	37268.0
7.0	6369.0	43637.0
8.0	6344.0	49981.0
9.0	6166.0	56147.0
... and 2 more rows		
(Total rows: 12)		

Q10: Compare the average order payment amount for orders placed with a promotion versus without a promotion. Show the promotion status label, average order value rounded to 2 decimal places, and total order count for each group.

PASS

Proposed SQL (Expected):

```
SELECT CASE WHEN o.promotion_id IS NOT NULL THEN 'With Promotion' ELSE 'Without Promotion' END AS promotion_status,
ROUND(AVG(pay.amount), 2) AS avg_order_value, COUNT(o.order_id) AS order_count FROM orders o JOIN payments pay ON o.order_id =
pay.order_id GROUP BY CASE WHEN o.promotion_id IS NOT NULL THEN 'With Promotion' ELSE 'Without Promotion' END
```

Proposed SQL Output (sorted):

promotion_status	avg_order_value	order_count

With Promotion	10060.27	300000
(Total rows: 1)		

Generated SQL (AI Output):

```
SELECT
CASE
WHEN o.promotion_id IS NULL
THEN 'Without Promotion'
ELSE 'With Promotion'
END AS promotion_status,
ROUND(AVG(p.amount), 2) AS average_order_value,
COUNT(o.order_id) AS total_order_count
FROM orders AS o
JOIN payments AS p
ON o.order_id = p.order_id
GROUP BY
promotion_status
ORDER BY
promotion_status;
```

Generated SQL Output (sorted):

promotion_status	average_order_value	total_order_count

With Promotion	10060.27	300000
(Total rows: 1)		

Q11: Calculate the median salary of employees for each store city.

PASS

Proposed SQL (Expected):

```
SELECT s.city, PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY e.salary) AS median_salary FROM employees e JOIN stores s ON e.store_id = s.store_id GROUP BY s.city ORDER BY median_salary DESC
```

Proposed SQL Output (sorted):

```
city | median_salary
-----
Bangalore | 45533.0
Delhi | 49096.0
Mumbai | 49680.0
Pune | 50219.0
(Total rows: 4)
```

Generated SQL (AI Output):

```
SELECT
s.city,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY e.salary) AS median_salary
FROM employees AS e
JOIN stores AS s
ON e.store_id = s.store_id
GROUP BY
s.city;
```

Generated SQL Output (sorted):

```
city | median_salary
-----
Bangalore | 45533.0
Delhi | 49096.0
Mumbai | 49680.0
Pune | 50219.0
(Total rows: 4)
```

Q12: Find all products whose price is above the average price of all products in their own category.
Show the product ID, its price, and the category name.

PASS

Proposed SQL (Expected):

```
SELECT p.product_id, p.price, c.category_name FROM products p JOIN categories c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(p2.price) FROM products p2 WHERE p2.category_id = p.category_id) ORDER BY c.category_name, p.price DESC
```

Proposed SQL Output (sorted):

```
product_id | price | category_name
-----
1136 | 4896.0 | Cat_1
1146 | 4815.0 | Cat_1
1823 | 4816.0 | Cat_1
2992 | 4851.0 | Cat_1
3615 | 4822.0 | Cat_1
3954 | 4857.0 | Cat_1
4281 | 4750.0 | Cat_1
4566 | 4923.0 | Cat_1
5739 | 4824.0 | Cat_1
721 | 4811.0 | Cat_1
... and 4979 more rows
(Total rows: 4989)
```

Generated SQL (AI Output):

```
WITH CategoryAverage AS (
  SELECT
    p.category_id,
    AVG(p.price) as avg_price
  FROM
    products p
  GROUP BY
    p.category_id
)
SELECT
  p.product_id,
  p.price,
  c.category_name
FROM
  products p
JOIN
  categories c ON p.category_id = c.category_id
JOIN
  CategoryAverage ca ON p.category_id = ca.category_id
WHERE
  p.price > ca.avg_price
ORDER BY
  c.category_name,
  p.price DESC;
```

Generated SQL Output (sorted):

```
product_id | price | category_name
-----
1136 | 4896.0 | Cat_1
1146 | 4815.0 | Cat_1
1823 | 4816.0 | Cat_1
2992 | 4851.0 | Cat_1
3615 | 4822.0 | Cat_1
3954 | 4857.0 | Cat_1
4281 | 4750.0 | Cat_1
4566 | 4923.0 | Cat_1
5739 | 4824.0 | Cat_1
721 | 4811.0 | Cat_1
... and 4979 more rows
(Total rows: 4989)
```

Q13: Compare total orders placed in 2022 vs 2023 and calculate the year-over-year percentage growth.

FAIL

Proposed SQL (Expected):

```
SELECT SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2022 THEN 1 ELSE 0 END) AS orders_2022, SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2023 THEN 1 ELSE 0 END) AS orders_2023, ROUND(100.0 * (SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2023 THEN 1 ELSE 0 END) - SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2022 THEN 1 ELSE 0 END)) / NULLIF(SUM(CASE WHEN EXTRACT(YEAR FROM order_date) = 2022 THEN 1 ELSE 0 END), 0), 2) AS yoy_growth_pct FROM orders
```

Proposed SQL Output (sorted):

```
orders_2022 | orders_2023 | yoy_growth_pct
-----
75124.0 | 75023.0 | -0.13
(Total rows: 1)
```

Generated SQL (AI Output):

```
WITH YearlyOrders AS (
  SELECT
    EXTRACT(YEAR FROM order_date) AS order_year,
    COUNT(order_id) AS total_orders
  FROM orders
  WHERE
    EXTRACT(YEAR FROM order_date) IN (2022, 2023)
  GROUP BY
    order_year
)
SELECT
  y2022.total_orders AS orders_2022,
  y2023.total_orders AS orders_2023,
  ((y2023.total_orders - y2022.total_orders) * 100.0) / y2022.total_orders AS yoy_growth_percentage
FROM
  YearlyOrders y2022,
  YearlyOrders y2023
WHERE
  y2022.order_year = 2022 AND y2023.order_year = 2023;
```

Generated SQL Output (sorted):

```
orders_2022 | orders_2023 | yoy_growth_percentage
-----
75124.0 | 75023.0 | -0.13444438528299876
(Total rows: 1)
```

Q14: Find all store cities that have never had any returns on orders placed from their store.

PASS

Proposed SQL (Expected):

```
SELECT s.city FROM stores s WHERE s.store_id NOT IN (SELECT DISTINCT o.store_id FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN returns r ON oi.order_item_id = r.order_item_id)
```

Proposed SQL Output (sorted):

(0 rows returned)

Generated SQL (AI Output):

```
SELECT DISTINCT s.city
FROM stores s
WHERE s.store_id NOT IN (
SELECT DISTINCT o.store_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN returns r ON oi.order_item_id = r.order_item_id
WHERE o.store_id IS NOT NULL
);
```

Generated SQL Output (sorted):

(0 rows returned)

Q15: Calculate the revenue contribution percentage of each product category to the overall total revenue. Show category name, total revenue, and percentage.

FAIL

Proposed SQL (Expected):

```
WITH category_revenue AS (SELECT c.category_name, SUM(oi.price * oi.qty) AS total_revenue FROM categories c JOIN products p ON c.category_id = p.category_id JOIN order_items oi ON p.product_id = oi.product_id GROUP BY c.category_name), grand_total AS (SELECT SUM(total_revenue) AS overall FROM category_revenue) SELECT cr.category_name, cr.total_revenue, ROUND(100.0 * cr.total_revenue / gt.overall, 2) AS revenue_pct FROM category_revenue cr, grand_total gt ORDER BY revenue_pct DESC
```

Proposed SQL Output (sorted):

category_name	total_revenue	revenue_pct

Cat_12	133897588.0	3.5
Cat_13	132409449.0	3.46
Cat_17	132292751.0	3.46
Cat_21	135329061.0	3.54
Cat_26	134697069.0	3.52
Cat_28	133603891.0	3.49
Cat_3	135379339.0	3.54
Cat_5	136704132.0	3.57
Cat_8	132816590.0	3.47
Cat_9	132465085.0	3.46
... and 20 more rows		
(Total rows: 30)		

Generated SQL (AI Output):

```
WITH CategoryRevenue AS (
SELECT
c.category_name,
SUM(oi.qty * oi.price) AS category_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
GROUP BY c.category_name
),
TotalRevenue AS (
SELECT SUM(category_revenue) as total_revenue
FROM CategoryRevenue
)
SELECT
cr.category_name,
cr.category_revenue,
(cr.category_revenue / tr.total_revenue) * 100 AS contribution_percentage
FROM CategoryRevenue cr, TotalRevenue tr
ORDER BY cr.category_revenue DESC;
```

Generated SQL Output (sorted):

category_name	category_revenue	contribution_percentage

Cat_12	133897588.0	3.498079110855642
Cat_13	132409449.0	3.4592014280855117
Cat_17	132292751.0	3.45615268880517
Cat_21	135329061.0	3.5354763924187265
Cat_26	134697069.0	3.5189655795919275
Cat_28	133603891.0	3.4904062665873723
Cat_3	135379339.0	3.5367899069051534
Cat_5	136704132.0	3.5714001697838826
Cat_8	132816590.0	3.469838000771742
Cat_9	132465085.0	3.460654920506985
... and 20 more rows		
(Total rows: 30)		

Q16: Find all customers who placed orders in at least 3 different months during 2023. Show the customer ID, city, and the count of active months.

PASS

Proposed SQL (Expected):

```
SELECT c.customer_id, c.city, COUNT(DISTINCT EXTRACT(MONTH FROM o.order_date)) AS active_months FROM customers c JOIN orders o
ON c.customer_id = o.customer_id WHERE EXTRACT(YEAR FROM o.order_date) = 2023 GROUP BY c.customer_id, c.city HAVING
COUNT(DISTINCT EXTRACT(MONTH FROM o.order_date)) >= 3 ORDER BY active_months DESC
```

Proposed SQL Output (sorted):

```
customer_id | city | active_months
-----
10452 | Pune | 6
16762 | Bangalore | 7
18877 | Bangalore | 7
22392 | Pune | 6
36623 | Delhi | 6
39882 | Bangalore | 7
45049 | Delhi | 6
5387 | Pune | 6
6368 | Pune | 7
7167 | Mumbai | 6
... and 7995 more rows
(Total rows: 8005)
```

Generated SQL (AI Output):

```
SELECT
c.customer_id,
c.city,
COUNT(DISTINCT EXTRACT(MONTH FROM o.order_date)) AS active_months
FROM
customers AS c
JOIN
orders AS o ON c.customer_id = o.customer_id
WHERE
EXTRACT(YEAR FROM o.order_date) = 2023
GROUP BY
c.customer_id,
c.city
HAVING
COUNT(DISTINCT EXTRACT(MONTH FROM o.order_date)) >= 3
ORDER BY
c.customer_id;
```

Generated SQL Output (sorted):

```
customer_id | city | active_months
-----
30 | Bangalore | 3
31 | Bangalore | 3
38 | Bangalore | 3
44 | Bangalore | 3
47 | Bangalore | 3
49 | Mumbai | 3
52 | Mumbai | 3
53 | Delhi | 3
54 | Mumbai | 4
6 | Bangalore | 3
... and 7995 more rows
(Total rows: 8005)
```

Q17: Find the store city with the second highest total revenue.

PASS

Proposed SQL (Expected):

```
WITH store_revenue AS (SELECT s.city, SUM(oi.price * oi.qty) AS total_revenue FROM stores s JOIN orders o ON s.store_id = o.store_id JOIN order_items oi ON o.order_id = oi.order_id GROUP BY s.city), ranked AS (SELECT city, total_revenue, DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS rnk FROM store_revenue) SELECT city, total_revenue FROM ranked WHERE rnk = 2
```

Proposed SQL Output (sorted):

```
city | total_revenue
-----
Pune | 1035744722.0
(Total rows: 1)
```

Generated SQL (AI Output):

```
WITH StoreRevenue AS (
SELECT
s.city,
SUM(oi.qty * oi.price) AS total_revenue
FROM stores AS s
JOIN orders AS o
ON s.store_id = o.store_id
JOIN order_items AS oi
ON o.order_id = oi.order_id
GROUP BY
s.city
)
SELECT
city,
total_revenue
FROM StoreRevenue
ORDER BY
total_revenue DESC
LIMIT 1
OFFSET 1;
```

Generated SQL Output (sorted):

```
city | total_revenue
-----
Pune | 1035744722.0
(Total rows: 1)
```

Q18: For each store city, calculate the ratio of total payments received to total revenue generated from order items. Show city, total payments, total revenue, and the ratio rounded to 4 decimal places.

PASS

Proposed SQL (Expected):

```
WITH store_payments AS (SELECT s.city, SUM(pay.amount) AS total_payments FROM stores s JOIN orders o ON s.store_id = o.store_id JOIN payments pay ON o.order_id = pay.order_id GROUP BY s.city), store_revenue AS (SELECT s.city, SUM(oi.price * oi.qty) AS total_revenue FROM stores s JOIN orders o ON s.store_id = o.store_id JOIN order_items oi ON o.order_id = oi.order_id GROUP BY s.city) SELECT sp.city, sp.total_payments, sr.total_revenue, ROUND(sp.total_payments / NULLIF(sr.total_revenue, 0), 4) AS payment_revenue_ratio FROM store_payments sp JOIN store_revenue sr ON sp.city = sr.city ORDER BY payment_revenue_ratio DESC
```

Proposed SQL Output (sorted):

city	total_payments	total_revenue	payment_revenue_ratio
Bangalore	629328486.0	796283132.0	0.7903
Delhi	634842800.0	805723746.0	0.7879
Mumbai	937497928.0	1189994536.0	0.7878
Pune	816411596.0	1035744722.0	0.7882
(Total rows: 4)			

Generated SQL (AI Output):

```
WITH StorePayments AS (
SELECT
s.city,
SUM(p.amount) AS total_payments
FROM stores s
JOIN orders o ON s.store_id = o.store_id
JOIN payments p ON o.order_id = p.order_id
GROUP BY s.city
),
StoreRevenue AS (
SELECT
s.city,
SUM(oi.qty * oi.price) AS total_revenue
FROM stores s
JOIN orders o ON s.store_id = o.store_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY s.city
)
SELECT
sp.city,
sp.total_payments,
sr.total_revenue,
ROUND(sp.total_payments / sr.total_revenue, 4) AS payment_to_revenue_ratio
FROM StorePayments sp
JOIN StoreRevenue sr ON sp.city = sr.city
ORDER BY sp.city;
```

Generated SQL Output (sorted):

city	total_payments	total_revenue	payment_to_revenue_ratio
Bangalore	629328486.0	796283132.0	0.7903
Delhi	634842800.0	805723746.0	0.7879
Mumbai	937497928.0	1189994536.0	0.7878
Pune	816411596.0	1035744722.0	0.7882
(Total rows: 4)			

Q19: Find all products that have been ordered by customers from more than 3 different cities. Show the product ID and the distinct city count.

PASS

Proposed SQL (Expected):

```
SELECT p.product_id, COUNT(DISTINCT c.city) AS city_count FROM products p JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id JOIN customers c ON o.customer_id = c.customer_id GROUP BY p.product_id HAVING
COUNT(DISTINCT c.city) > 3 ORDER BY city_count DESC
```

Proposed SQL Output (sorted):

product_id	city_count
1	4
10	4
2	4
3	4
4	4
5	4
6	4
7	4
8	4
9	4
... and 9990 more rows	
(Total rows: 10000)	

Generated SQL (AI Output):

```
SELECT
p.product_id,
COUNT(DISTINCT c.city) AS distinct_city_count
FROM
order_items oi
JOIN
orders o ON oi.order_id = o.order_id
JOIN
customers c ON o.customer_id = c.customer_id
JOIN
products p ON oi.product_id = p.product_id
GROUP BY
p.product_id
HAVING
COUNT(DISTINCT c.city) > 3
ORDER BY
distinct_city_count DESC;
```

Generated SQL Output (sorted):

product_id	distinct_city_count
1	4
10	4
2	4
3	4
4	4
5	4
6	4
7	4
8	4
9	4
... and 9990 more rows	
(Total rows: 10000)	

Q20: For each month of 2023, identify which product category generated the highest sales revenue.

Show the month, category name, and total sales.

PASS

Proposed SQL (Expected):

```
WITH monthly_category_sales AS (SELECT EXTRACT(MONTH FROM o.order_date) AS month, c.category_name, SUM(oi.price * oi.qty) AS total_sales FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id = p.product_id JOIN categories c ON p.category_id = c.category_id WHERE EXTRACT(YEAR FROM o.order_date) = 2023 GROUP BY month, c.category_name), ranked AS (SELECT month, category_name, total_sales, ROW_NUMBER() OVER (PARTITION BY month ORDER BY total_sales DESC) AS rn FROM monthly_category_sales) SELECT month, category_name, total_sales FROM ranked WHERE rn = 1 ORDER BY month
```

Proposed SQL Output (sorted):

```
month | category_name | total_sales
-----
1.0 | Cat_12 | 3244665.0
10.0 | Cat_5 | 3178269.0
2.0 | Cat_8 | 2896349.0
3.0 | Cat_15 | 2991301.0
4.0 | Cat_26 | 2945152.0
5.0 | Cat_16 | 3110061.0
6.0 | Cat_10 | 3078597.0
7.0 | Cat_3 | 3106950.0
8.0 | Cat_5 | 3188994.0
9.0 | Cat_28 | 3084101.0
... and 2 more rows
(Total rows: 12)
```

Generated SQL (AI Output):

```
WITH MonthlyCategorySales AS (
SELECT
EXTRACT(MONTH FROM o.order_date) AS sales_month,
c.category_name,
SUM(oi.qty * oi.price) AS total_sales
FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON oi.product_id = p.product_id
JOIN categories AS c ON p.category_id = c.category_id
WHERE EXTRACT(YEAR FROM o.order_date) = 2023
GROUP BY sales_month, c.category_name
),
RankedSales AS (
SELECT
sales_month,
category_name,
total_sales,
ROW_NUMBER() OVER(PARTITION BY sales_month ORDER BY total_sales DESC) as rn
FROM MonthlyCategorySales
)
SELECT
sales_month,
category_name,
total_sales
FROM RankedSales
WHERE rn = 1
ORDER BY sales_month;
```

Generated SQL Output (sorted):

```
sales_month | category_name | total_sales
-----
1.0 | Cat_12 | 3244665.0
10.0 | Cat_5 | 3178269.0
2.0 | Cat_8 | 2896349.0
3.0 | Cat_15 | 2991301.0
4.0 | Cat_26 | 2945152.0
5.0 | Cat_16 | 3110061.0
6.0 | Cat_10 | 3078597.0
7.0 | Cat_3 | 3106950.0
8.0 | Cat_5 | 3188994.0
9.0 | Cat_28 | 3084101.0
... and 2 more rows
(Total rows: 12)
```

Q21: Find customers whose very first order ever placed used a promotion. Show the customer ID and city.

PASS

Proposed SQL (Expected):

```
WITH first_orders AS (SELECT customer_id, MIN(order_date) AS first_order_date FROM orders GROUP BY customer_id) SELECT
c.customer_id, c.city FROM customers c JOIN first_orders fo ON c.customer_id = fo.customer_id JOIN orders o ON c.customer_id =
o.customer_id AND o.order_date = fo.first_order_date WHERE o.promotion_id IS NOT NULL
```

Proposed SQL Output (sorted):

```
customer_id | city
-----
12576 | Pune
17072 | Pune
35091 | Delhi
35637 | Pune
3757 | Bangalore
41037 | Bangalore
43864 | Mumbai
46646 | Pune
47641 | Pune
49434 | Bangalore
... and 49958 more rows
(Total rows: 49968)
```

Generated SQL (AI Output):

```
WITH RankedOrders AS (
SELECT
customer_id,
order_id,
promotion_id,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date ASC) as rn
FROM
orders
)
SELECT
c.customer_id,
c.city
FROM
customers c
JOIN
RankedOrders ro ON c.customer_id = ro.customer_id
WHERE
ro.rn = 1
AND ro.promotion_id IS NOT NULL;
```

Generated SQL Output (sorted):

```
customer_id | city
-----
1 | Mumbai
10 | Bangalore
2 | Bangalore
3 | Pune
4 | Mumbai
5 | Delhi
6 | Bangalore
7 | Delhi
8 | Bangalore
9 | Mumbai
... and 49876 more rows
(Total rows: 49886)
```

Q22: Find all employees who earn more than the average salary of all employees in their own store city.
Show employee ID, city, their salary, and the city average salary.

PASS

Proposed SQL (Expected):

```
WITH city_avg AS (SELECT s.city, AVG(e.salary) AS avg_salary FROM employees e JOIN stores s ON e.store_id = s.store_id GROUP BY s.city) SELECT e.employee_id, s.city, e.salary, ROUND(ca.avg_salary, 2) AS city_avg_salary FROM employees e JOIN stores s ON e.store_id = s.store_id JOIN city_avg ca ON s.city = ca.city WHERE e.salary > ca.avg_salary ORDER BY s.city, e.salary DESC
```

Proposed SQL Output (sorted):

```
employee_id | city | salary | city_avg_salary
-----
112 | Bangalore | 79343.0 | 48171.75
131 | Bangalore | 77840.0 | 48171.75
335 | Bangalore | 76914.0 | 48171.75
340 | Bangalore | 77875.0 | 48171.75
426 | Bangalore | 79297.0 | 48171.75
554 | Bangalore | 77292.0 | 48171.75
576 | Bangalore | 76700.0 | 48171.75
812 | Bangalore | 79495.0 | 48171.75
937 | Bangalore | 76703.0 | 48171.75
942 | Bangalore | 79820.0 | 48171.75
... and 481 more rows
(Total rows: 491)
```

Generated SQL (AI Output):

```
WITH CityAvgSalary AS (
SELECT
s.city,
AVG(e.salary) AS city_avg_salary
FROM employees AS e
JOIN stores AS s
ON e.store_id = s.store_id
GROUP BY
s.city
)
SELECT
e.employee_id,
s.city,
e.salary,
ROUND(cas.city_avg_salary, 2) AS city_average_salary
FROM employees AS e
JOIN stores AS s
ON e.store_id = s.store_id
JOIN CityAvgSalary AS cas
ON s.city = cas.city
WHERE
e.salary > cas.city_avg_salary
ORDER BY
s.city,
e.salary DESC;
```

Generated SQL Output (sorted):

```
employee_id | city | salary | city_average_salary
-----
112 | Bangalore | 79343.0 | 48171.75
131 | Bangalore | 77840.0 | 48171.75
335 | Bangalore | 76914.0 | 48171.75
340 | Bangalore | 77875.0 | 48171.75
426 | Bangalore | 79297.0 | 48171.75
554 | Bangalore | 77292.0 | 48171.75
576 | Bangalore | 76700.0 | 48171.75
812 | Bangalore | 79495.0 | 48171.75
937 | Bangalore | 76703.0 | 48171.75
942 | Bangalore | 79820.0 | 48171.75
... and 481 more rows
(Total rows: 491)
```

Q23: Calculate the total revenue lost to refunds for each supplier country. Show the country and the total refund amount, ordered from highest to lowest.

PASS

Proposed SQL (Expected):

```
SELECT s.country, SUM(r.refund) AS total_refunds_lost FROM suppliers s JOIN products p ON s.supplier_id = p.supplier_id JOIN
order_items oi ON p.product_id = oi.product_id JOIN returns r ON oi.order_item_id = r.order_item_id GROUP BY s.country ORDER
BY total_refunds_lost DESC
```

Proposed SQL Output (sorted):

country	total_refunds_lost
China	24117657.0
India	23418481.0
USA	28426162.0

(Total rows: 3)

Generated SQL (AI Output):

```
SELECT
s.country,
SUM(r.refund) AS total_refund_amount
FROM returns AS r
JOIN order_items AS oi
ON r.order_item_id = oi.order_item_id
JOIN products AS p
ON oi.product_id = p.product_id
JOIN suppliers AS s
ON p.supplier_id = s.supplier_id
GROUP BY
s.country
ORDER BY
total_refund_amount DESC;
```

Generated SQL Output (sorted):

country	total_refund_amount
China	24117657.0
India	23418481.0
USA	28426162.0

(Total rows: 3)

Q24: Find all store cities where every single order placed from that store had a promotion applied. Show only cities that have at least one order.

PASS

Proposed SQL (Expected):

```
SELECT DISTINCT s.city FROM stores s WHERE EXISTS (SELECT 1 FROM orders o WHERE o.store_id = s.store_id) AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.store_id = s.store_id AND o.promotion_id IS NULL)
```

Proposed SQL Output (sorted):

```
city
----
Bangalore
Delhi
Mumbai
Pune
(Total rows: 4)
```

Generated SQL (AI Output):

```
SELECT
s.city
FROM stores AS s
JOIN orders AS o
ON s.store_id = o.store_id
GROUP BY
s.city
HAVING
COUNT(o.order_id) = COUNT(CASE WHEN o.promotion_id IS NOT NULL THEN 1 END) AND COUNT(o.order_id) > 0;
```

Generated SQL Output (sorted):

```
city
----
Bangalore
Delhi
Mumbai
Pune
(Total rows: 4)
```

Q25: For each store city, calculate the average number of items (total quantity) per order. Show the city and the average rounded to 2 decimal places.

PASS

Proposed SQL (Expected):

```
WITH order_totals AS (SELECT oi.order_id, SUM(oi.qty) AS total_items FROM order_items oi GROUP BY oi.order_id) SELECT s.city,
ROUND(AVG(ot.total_items), 2) AS avg_items_per_order FROM stores s JOIN orders o ON s.store_id = o.store_id JOIN order_totals
ot ON o.order_id = ot.order_id GROUP BY s.city ORDER BY avg_items_per_order DESC
```

Proposed SQL Output (sorted):

```
city | avg_items_per_order
-----
Bangalore | 5.76
Delhi | 5.81
Mumbai | 5.8
Pune | 5.78
(Total rows: 4)
```

Generated SQL (AI Output):

```
WITH order_quantities AS (
SELECT
order_id,
SUM(qty) AS total_quantity
FROM order_items
GROUP BY
order_id
)
SELECT
s.city,
ROUND(AVG(oq.total_quantity), 2) AS average_items_per_order
FROM stores AS s
JOIN orders AS o
ON s.store_id = o.store_id
JOIN order_quantities AS oq
ON o.order_id = oq.order_id
GROUP BY
s.city
ORDER BY
s.city;
```

Generated SQL Output (sorted):

```
city | average_items_per_order
-----
Bangalore | 5.76
Delhi | 5.81
Mumbai | 5.8
Pune | 5.78
(Total rows: 4)
```

Q26: For each customer city, find the most recent order date and calculate the total revenue generated on that specific date in that city.

PASS

Proposed SQL (Expected):

```
WITH latest_per_city AS (SELECT c.city, MAX(o.order_date) AS latest_date FROM customers c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.city) SELECT lpc.city, lpc.latest_date, SUM(oi.price * oi.qty) AS revenue_on_latest_date FROM latest_per_city lpc JOIN customers c ON c.city = lpc.city JOIN orders o ON c.customer_id = o.customer_id AND o.order_date = lpc.latest_date JOIN order_items oi ON o.order_id = oi.order_id GROUP BY lpc.city, lpc.latest_date ORDER BY lpc.latest_date DESC
```

Proposed SQL Output (sorted):

city	latest_date	revenue_on_latest_date

Bangalore	2024-01-01	554313.0
Delhi	2024-01-01	697572.0
Mumbai	2024-01-01	523277.0
Pune	2024-01-01	702298.0
(Total rows: 4)		

Generated SQL (AI Output):

```
WITH LastOrderDateByCity AS (
SELECT
c.city,
MAX(o.order_date) AS max_order_date
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
GROUP BY c.city
)
SELECT
c.city,
o.order_date,
SUM(oi.qty * oi.price) AS total_revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN LastOrderDateByCity AS lod ON c.city = lod.city AND o.order_date = lod.max_order_date
GROUP BY c.city, o.order_date
ORDER BY c.city;
```

Generated SQL Output (sorted):

city	order_date	total_revenue

Bangalore	2024-01-01	554313.0
Delhi	2024-01-01	697572.0
Mumbai	2024-01-01	523277.0
Pune	2024-01-01	702298.0
(Total rows: 4)		

Q27: Find all product categories for which not a single item has ever been returned. Show the category name.

PASS

Proposed SQL (Expected):

```
SELECT c.category_name FROM categories c WHERE NOT EXISTS (SELECT 1 FROM products p JOIN order_items oi ON p.product_id = oi.product_id JOIN returns r ON oi.order_item_id = r.order_item_id WHERE p.category_id = c.category_id)
```

Proposed SQL Output (sorted):

(0 rows returned)

Generated SQL (AI Output):

```
SELECT c.category_name
FROM categories c
WHERE NOT EXISTS (
  SELECT 1
  FROM products p
  JOIN order_items oi ON p.product_id = oi.product_id
  JOIN returns r ON oi.order_item_id = r.order_item_id
  WHERE p.category_id = c.category_id
);
```

Generated SQL Output (sorted):

(0 rows returned)

Q28: For each store city, calculate the ratio of delivered shipments to non-delivered shipments. Show city, delivered count, non-delivered count, and the delivery ratio rounded to 2 decimal places.

FAIL

Proposed SQL (Expected):

```
SELECT s.city, SUM(CASE WHEN sh.status = 'delivered' THEN 1 ELSE 0 END) AS delivered, SUM(CASE WHEN sh.status != 'delivered' THEN 1 ELSE 0 END) AS not_delivered, ROUND(1.0 * SUM(CASE WHEN sh.status = 'delivered' THEN 1 ELSE 0 END) / NULLIF(SUM(CASE WHEN sh.status != 'delivered' THEN 1 ELSE 0 END), 0), 2) AS delivery_ratio FROM stores s JOIN orders o ON s.store_id = o.store_id JOIN shipments sh ON o.order_id = sh.order_id GROUP BY s.city ORDER BY delivery_ratio DESC
```

Proposed SQL Output (sorted):

```
city | delivered | not_delivered | delivery_ratio
-----
Bangalore | 21092 | 41663 | 0.51
Delhi | 21179 | 41978 | 0.5
Mumbai | 30651 | 62320 | 0.49
Pune | 26940 | 54177 | 0.5
(Total rows: 4)
```

Generated SQL (AI Output):

```
WITH status_counts AS (
  SELECT
    s.city,
    COUNT(CASE WHEN sh.status = 'Delivered' THEN 1 END) AS delivered_count,
    COUNT(CASE WHEN sh.status != 'Delivered' THEN 1 END) AS non_delivered_count
  FROM stores AS s
  JOIN orders AS o
  ON s.store_id = o.store_id
  JOIN shipments AS sh
  ON o.order_id = sh.order_id
  GROUP BY
    s.city
)
SELECT
  city,
  delivered_count,
  non_delivered_count,
  ROUND(
    CAST(delivered_count AS NUMERIC) / NULLIF(non_delivered_count, 0),
    2
  ) AS delivery_ratio
FROM status_counts
ORDER BY
  city;
```

Generated SQL Output (sorted):

```
city | delivered_count | non_delivered_count | delivery_ratio
-----
Bangalore | 0 | 62755 | 0.0
Delhi | 0 | 63157 | 0.0
Mumbai | 0 | 92971 | 0.0
Pune | 0 | 81117 | 0.0
(Total rows: 4)
```

Q29: Find all customers who have placed at least one order but have never had any of their order items returned. Show customer ID and city.

PASS

Proposed SQL (Expected):

```
SELECT c.customer_id, c.city FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id) AND NOT EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN returns r ON oi.order_item_id = r.order_item_id WHERE o.customer_id = c.customer_id)
```

Proposed SQL Output (sorted):

```
customer_id | city
-----
10295 | Delhi
11890 | Delhi
2026 | Mumbai
22262 | Mumbai
2848 | Mumbai
36761 | Mumbai
40448 | Delhi
46770 | Delhi
48394 | Delhi
6122 | Delhi
... and 28499 more rows
(Total rows: 28509)
```

Generated SQL (AI Output):

```
SELECT c.customer_id, c.city
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN returns r ON oi.order_item_id = r.order_item_id
GROUP BY c.customer_id, c.city
HAVING COUNT(r.return_id) = 0
ORDER BY c.customer_id;
```

Generated SQL Output (sorted):

```
customer_id | city
-----
10 | Bangalore
11 | Delhi
12 | Pune
14 | Bangalore
15 | Pune
18 | Bangalore
19 | Delhi
6 | Bangalore
7 | Delhi
9 | Mumbai
... and 28499 more rows
(Total rows: 28509)
```

Q30: For each quarter of 2023 (Q1=Jan-Mar, Q2=Apr-Jun, Q3=Jul-Sep, Q4=Oct-Dec), show total revenue, total refunds, and net revenue (revenue minus refunds).

FAIL

Proposed SQL (Expected):

```
SELECT CASE WHEN EXTRACT(MONTH FROM o.order_date) BETWEEN 1 AND 3 THEN 'Q1' WHEN EXTRACT(MONTH FROM o.order_date) BETWEEN 4 AND 6 THEN 'Q2' WHEN EXTRACT(MONTH FROM o.order_date) BETWEEN 7 AND 9 THEN 'Q3' ELSE 'Q4' END AS quarter, SUM(oi.price * oi.qty) AS total_revenue, COALESCE(SUM(r.refund), 0) AS total_refunds, SUM(oi.price * oi.qty) - COALESCE(SUM(r.refund), 0) AS net_revenue FROM orders o JOIN order_items oi ON o.order_id = oi.order_id LEFT JOIN returns r ON oi.order_item_id = r.order_item_id WHERE EXTRACT(YEAR FROM o.order_date) = 2023 GROUP BY quarter ORDER BY quarter
```

Proposed SQL Output (sorted):

quarter	total_revenue	total_refunds	net_revenue
Q1	236734044.0	4683037.0	232051007.0
Q2	237954576.0	4781722.0	233172854.0
Q3	242524228.0	4824444.0	237699784.0
Q4	241992816.0	4567233.0	237425583.0
(Total rows: 4)			

Generated SQL (AI Output):

```
WITH quarterly_revenue AS (
  SELECT
    CASE
      WHEN EXTRACT(MONTH FROM o.order_date) IN (1, 2, 3)
      THEN 'Q1'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (4, 5, 6)
      THEN 'Q2'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (7, 8, 9)
      THEN 'Q3'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (10, 11, 12)
      THEN 'Q4'
    END AS quarter,
    SUM(p.amount) AS total_revenue
  FROM orders AS o
  JOIN payments AS p
  ON o.order_id = p.order_id
  WHERE EXTRACT(YEAR FROM o.order_date) = 2023
  GROUP BY
    quarter
), quarterly_refunds AS (
  SELECT
    CASE
      WHEN EXTRACT(MONTH FROM o.order_date) IN (1, 2, 3)
      THEN 'Q1'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (4, 5, 6)
      THEN 'Q2'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (7, 8, 9)
      THEN 'Q3'
      WHEN EXTRACT(MONTH FROM o.order_date) IN (10, 11, 12)
      THEN 'Q4'
    END AS quarter,
    SUM(r.refund) AS total_refunds
  FROM orders AS o
  JOIN order_items AS oi
  ON o.order_id = oi.order_id
  JOIN returns AS r
  ON oi.order_item_id = r.order_item_id
  WHERE EXTRACT(YEAR FROM o.order_date) = 2023
  GROUP BY
    quarter
)
SELECT
  qr.quarter,
  qr.total_revenue,
  COALESCE(qrf.total_refunds, 0) AS total_refunds,
  qr.total_revenue - COALESCE(qrf.total_refunds, 0) AS net_revenue
FROM quarterly_revenue AS qr
LEFT JOIN quarterly_refunds AS qrf
ON qr.quarter = qrf.quarter
ORDER BY
  qr.quarter;
```

Generated SQL Output (sorted):

quarter	total_revenue	total_refunds	net_revenue
Q1	186555706.0	4683037.0	181872669.0
Q2	188166744.0	4781722.0	183385022.0

Q3		189347231.0		4824444.0		184522787.0
Q4		189852525.0		4567233.0		185285292.0
(Total rows: 4)						

Q31: Calculate a rolling 3-month average of monthly order counts for the year 2023. Show month, that month's order count, and the 3-month rolling average.

FAIL

Proposed SQL (Expected):

```
WITH monthly_orders AS (SELECT EXTRACT(MONTH FROM order_date) AS month, COUNT(order_id) AS order_count FROM orders WHERE
EXTRACT(YEAR FROM order_date) = 2023 GROUP BY month) SELECT month, order_count, ROUND(AVG(order_count) OVER (ORDER BY month
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS rolling_3m_avg FROM monthly_orders ORDER BY month
```

Proposed SQL Output (sorted):

month	order_count	rolling_3m_avg
1.0	6369.0	6369.0
10.0	6350.0	6286.67
2.0	5769.0	6069.0
3.0	6402.0	6180.0
4.0	6142.0	6104.33
5.0	6450.0	6331.33
6.0	6136.0	6242.67
7.0	6369.0	6318.33
8.0	6344.0	6283.0
9.0	6166.0	6293.0
... and 2 more rows		
(Total rows: 12)		

Generated SQL (AI Output):

```
WITH monthly_orders AS (
SELECT
EXTRACT(MONTH FROM order_date) AS month,
TO_CHAR(order_date, 'YYYY-MM') AS month_year,
COUNT(order_id) AS monthly_count
FROM orders
WHERE
EXTRACT(YEAR FROM order_date) = 2023
GROUP BY
1, 2
)
SELECT
month_year,
monthly_count,
ROUND(AVG(monthly_count) OVER (ORDER BY month_year ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS rolling_3_month_avg
FROM monthly_orders
ORDER BY
month_year;
```

Generated SQL Output (sorted):

month_year	monthly_count	rolling_3_month_avg
2023-01	6369	6369.0
2023-02	5769	6069.0
2023-03	6402	6180.0
2023-04	6142	6104.33
2023-05	6450	6331.33
2023-06	6136	6242.67
2023-07	6369	6318.33
2023-08	6344	6283.0
2023-09	6166	6293.0
2023-10	6350	6286.67
... and 2 more rows		
(Total rows: 12)		